Event Sourcing & CQRS

vent Sourcing/CQRS · Bank account ledger + balance read model · Build with Waq

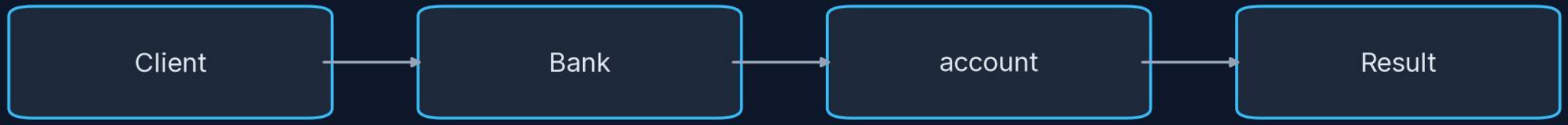
Today's Hook

- Problem: Bank account ledger + balance read model
- Pattern: Event Sourcing/CQRS — earned from the pain, not trivia.
- Runnable main on JDK 25 with captured stdout.
- Naive → solution → failure paths → trade-offs.

The Problem

- Bank account ledger + balance read model
- Naive design shows the cost clearly.
- Pattern isolates the force behind a clean boundary.
- Tests cover happy path and injected failures.

Structure



Event Sourcing/CQRS: Bank account ledger + balance read model

Real-World Scenario

- Running example: Bank account ledger + balance read model
- Same names in naive code, refactor, and Demo output.
- No shared StreamForge framing unless it fits.
- Product language first; pattern vocabulary second.

Naive Code

```
package com.buildwithwaqar.patterns.enterprise.eventsourcing;

import com.buildwithwaqar.patterns.enterprise.eventsourcing.*;
/** Problem: reconstruct account balance from an append-only ledger. */
public final class UseCaseDemo {
  public static void main(String[] a) {
    System.out.println("=== Event Sourcing problem: bank ledger ===");
    var acct = new Account();
    acct.deposit(100); acct.withdraw(30);
    System.out.println("balance=" + acct.balance());
    System.out.println("events=" + acct.events());
    var view = new BalanceView();
    acct.events().forEach(view::project);
    System.out.println("readModel=" + view.balance());
  }
}
```

Why Naive Hurts

- Call sites couple to the wrong shape or leak on partial failure.
- Parameter drilling / duplicated envelopes / missing rollback.
- Hard to unit-test without the real dependency.
- New variants require shotgun edits.

Solution

```java
package com.buildwithwaqar.patterns.enterprise.eventsourcing;

import java.util.ArrayList;
import java.util.List;

sealed interface AccountEvent permits Deposited, Withdrawn {}
record Deposited(int amount) implements AccountEvent {}
record Withdrawn(int amount) implements AccountEvent {}

/** Minimal event-sourced aggregate + separate read model (CQRS). */
public final class Account {
    private final List<AccountEvent> events = new ArrayList<>();
    private int balance;

    public void deposit(int amount) {
        apply(new Deposited(amount));
    }

    public void withdraw(int amount) {
        if (amount > balance) throw new IllegalStateException("insufficient");
        apply(new Withdrawn(amount));
    }
```

Solution (2)

```
    private void apply(AccountEvent e) {
        switch (e) {
            case Deposited(var a) -> balance += a;
            case Withdrawn(var a) -> balance -= a;
        }
        events.add(e);
    }

    public int balance() { return balance; }
    public List<AccountEvent> events() { return List.copyOf(events); }
}

/** Read model projected from events. */
final class BalanceView {
    private int balance;
    void project(AccountEvent e) {
        switch (e) {
            case Deposited(var a) -> balance += a;
            case Withdrawn(var a) -> balance -= a;
        }
    }
```

Solution (3)

```
    int balance() { return balance; }
}
```

Runnable main

```
package com.buildwithwaqar.patterns.enterprise.eventsourcing;

import com.buildwithwaqar.patterns.enterprise.eventsourcing.*;
/** Problem: reconstruct account balance from an append-only ledger. */
public final class UseCaseDemo {
  public static void main(String[] a) {
    System.out.println("=== Event Sourcing problem: bank ledger ===");
    var acct = new Account();
    acct.deposit(100); acct.withdraw(30);
    System.out.println("balance=" + acct.balance());
    System.out.println("events=" + acct.events());
    var view = new BalanceView();
    acct.events().forEach(view::project);
    System.out.println("readModel=" + view.balance());
  }
}
```

Walkthrough Output

```
=== Event Sourcing problem: bank ledger ===
balance=70
events=[Deposited[amount=100], Withdrawn[amount=30]]
readModel=70
```

Step-by-Step Trace

- Each stdout line maps to a stage above.
- Re-run the main class locally on JDK 25.
- Failure-path lines prove compensations / rejects / missing files.
- Pause and predict the next line before revealing.

Modern Java 25 / Zazr

```
// Java 25 / Zazr direction
// records + sealed + switch
// inject Env / collaborators at the edge
// virtual threads for blocking fan-out
```

Trade-offs

- Clarity vs ceremony — refuse five types for two lines.
- Teaching demos omit production metrics/idempotency keys.
- Prefer simpler language features when they erase the force.
- Know the twin pattern often confused with Event Sourcing/CQRS.

Related Patterns

- Contrast Event Sourcing/CQRS with its neighbor pattern.
- Name the roles from memory in sixty seconds.
- Point to the season map for what comes next.
- Link the GitHub episode folder in the description.

Recap

- Problem → naive → Event Sourcing/CQRS → Demo stdout.
- Failure paths are part of the lesson.
- Teach when NOT to use it.
- Practice: break a test, fix it, re-run the main.